Dijkstra Shortest Path and Tour Planning

Clearance-weighted grid Dijkstra, Held-Karp ordering of multi-stop deliveries, and measured results on the SLAM map

Hüseyin Tozluyurt · ROS 2 Jazzy / Gazebo Harmonic simulation stack · November 2026

1. Two shortest-path problems

A delivery mission has two layers. **Where to drive between two points** is a shortest-path problem on the map grid, solved with **Dijkstra's algorithm**. **In which order to visit several delivery points** is a travelling-salesman problem on top of it, solved exactly with **Held-Karp dynamic programming** on the Dijkstra distance matrix. Both run in `delivery_navigator.py` and behind the web app's `/robot/plan` and `/robot/mission`.

291 → 176	−39 %	132 ms	8 ms
tour cost: arrival order → optimal (6 stops)	driving saved	one full Dijkstra tree, 230×360 grid	exact order for 6 stops

2. From SLAM map to planning graph

1. The saved 0.1 m occupancy grid is down-sampled to **0.2 m cells** (here 230×360 cells for campus_full_map7).
2. A distance transform on the *full-resolution* map gives every cell its **clearance** – distance to the nearest wall – taking the worst point inside the cell, so thin or doubled walls do not eat corridor cells.
3. Cells with clearance below the **robot radius (0.45 m)**, and unknown cells, are blocked.
4. Free cells get a cost $1 + w \cdot t^2$ with $w = 4$ and t rising from 0 at 1.0 m clearance to 1 at the robot radius. The cheapest path therefore keeps to the middle of corridors and doors instead of grazing walls.
5. Each cell is connected to its 8 neighbours; an edge costs step length × mean cost of the two cells (1 or √2 times 0.2 m).

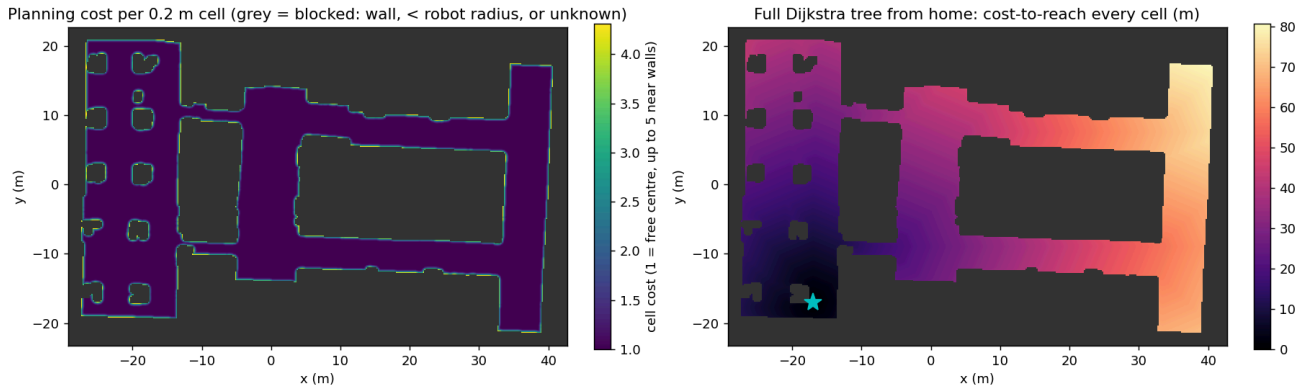


Figure 1. campus_full_map7. Left: per-cell planning cost (bright = close to a wall, dark grey = blocked). Right: one full Dijkstra run from home gives the cost-to-reach of every cell – this single tree answers "how far to any delivery point" at once.

3. Dijkstra's algorithm as implemented

```
g[start] = 0, all other g = ∞; push (0, start) on a binary heap
while heap not empty:
    pop cell i with the smallest g; skip if already closed; close it
    for each of the 8 neighbours j (not blocked, not closed):
        ng = g[i] + step(i,j) · ½(cost[i] + cost[j])
        if ng < g[j]: g[j] = ng; parent[j] = i; push (ng, j)
path = follow parent[] back from the goal
```

`PlanningGrid._search` uses Python's `heapq` with lazy deletion, so the run time is O(E log V) on the grid. With a goal and the octile heuristic the same function becomes A*; the navigator uses the pure Dijkstra form because one run from a point returns the whole tree (`dijkstra_tree`), from which the cost and path to *every* stop are read without searching again.

Same query: home → b_delivery_point_3	Time	Path length
Dijkstra (no heuristic, stops at goal)	115 ms	71.5 m
A* (octile heuristic)	56 ms	71.5 m

Both return the identical path; A* explores fewer cells for one goal, but for tours the full Dijkstra tree is cheaper than one A* per pair of points.

Robustness rules around the planner

- Replan from the current pose every 15 s and after every pose correction.
- A goal too close to a wall or table is moved to the nearest reachable cell with room (logged as "stopped x m before").
- Lidar obstacles are added with their own clearance for 30 s; if no path is found the planner relaxes step by step (drop remembered obstacles → 0.1 m less margin → allow unknown cells).
- The path is followed with pure pursuit; an obstacle closer than the stop distance halts the robot and, if it stays more than 2 s, is put into the map and planned around.

4. Visiting order: Held-Karp

For a mission with start A, stops $x_1 \ldots x_k$ and end B, one Dijkstra tree from A and from every stop fills the (k+2)×(k+2) matrix of real driving costs. Held-Karp then finds the cheapest open path A → all stops → B exactly:

```
best[{x_i}, i] = D[A, x_i]
best[S ∪ {j}, j] = min over i in S of best[S, i] + D[x_i, x_j]
answer = min over i of best[all, i] + D[x_i, B]   (back-pointers give the order)
```

This is $O(k^2 \cdot 2^k)$ instead of O(k!) for trying every order. The navigator uses it up to 12 stops and switches to nearest-neighbour + 2-opt above that.

5. Experiment on the final map

Six delivery points of campus_full_map7, start and end at home, listed in "order of arrival": `a_delivery_point_2`, `b_delivery_point_3`, `r_delivery_point_1`, `b_delivery_point_2`, `a_delivery_point_7`, `c_table_north`. The planner and ordering code were imported unchanged from the package and run offline on the saved map (script `reports/src/scripts/dij.py`).

Method	Tour cost	vs. optimum
Order of arrival (no optimisation)	290.7	+65 %
Nearest neighbour	176.5	+0.3 %
Nearest neighbour + 2-opt	176.0	+0.0 %
Held-Karp (used)	**176.0**	optimal
Brute force over all 720 orders (check)	176.0	same as Held-Karp

Optimal order: home → a_delivery_point_2 → a_delivery_point_7 → c_table_north → r_delivery_point_1 → b_delivery_point_3 → b_delivery_point_2 → home. Costs are in cost-weighted metres, i.e. path length inflated slightly near walls, so they are a little above the geometric length.

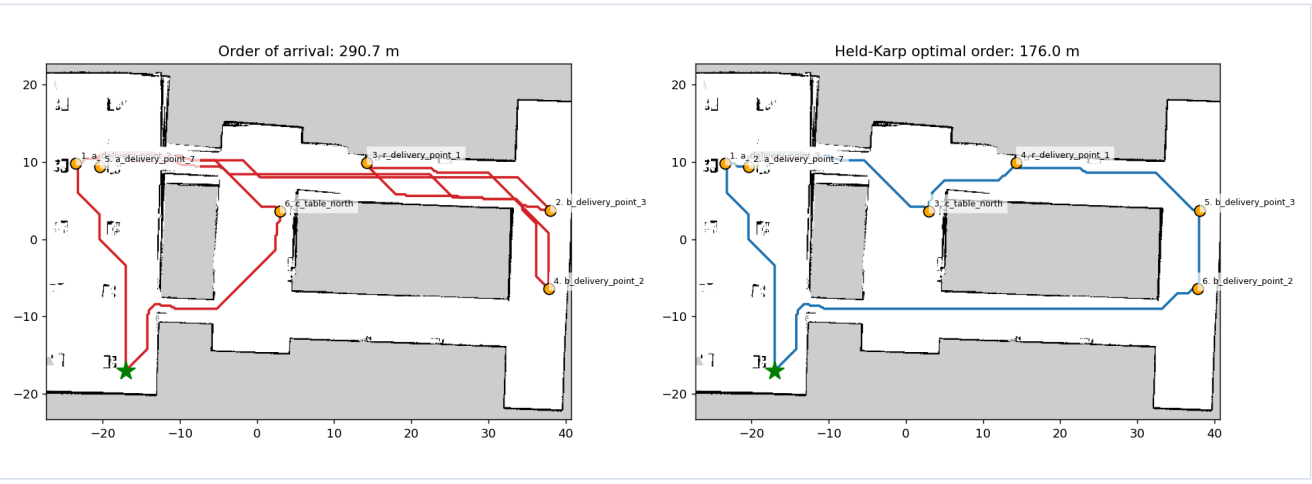


Figure 2. The same six stops driven in arrival order (left) and in the Held-Karp order (right), each leg drawn from the Dijkstra trees on the SLAM map.

Why a map-based distance is needed

From home to	Dijkstra cost	Straight line (m)	Ratio
`a_delivery_point_2`	29.5	27.6	1.07
`b_delivery_point_3`	71.9	58.8	1.22
`r_delivery_point_1`	50.2	41.2	1.22
`b_delivery_point_2`	62.9	55.8	1.13
`a_delivery_point_7`	27.8	26.6	1.05
`c_table_north`	34.8	28.7	1.21

Walls make the real distance 5–25 % longer than the straight line, and not by the same factor for every point, so ordering by straight-line distance can choose the wrong tour.

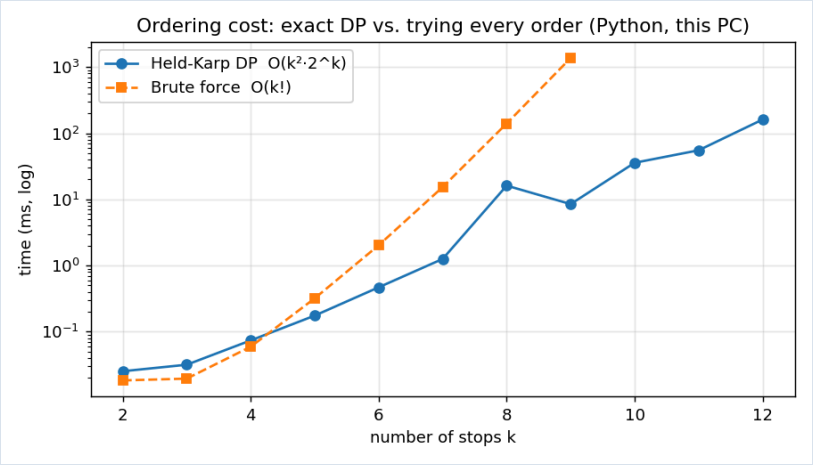


Figure 3. Run time of the exact ordering. Held-Karp needs 161 ms for 12 stops; brute force already takes 1.4 s for 9 stops and would take hours for 12.

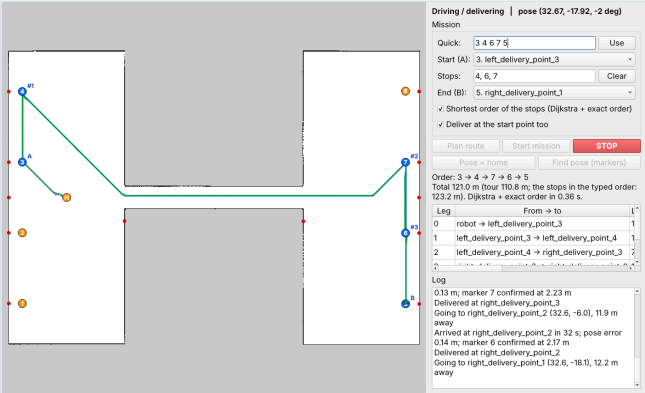


Figure 4. Delivery GUI: a multi-stop mission typed as "3 4 6 7 5"; the GUI shows the shortest order, each leg and the total compared with the typed order.

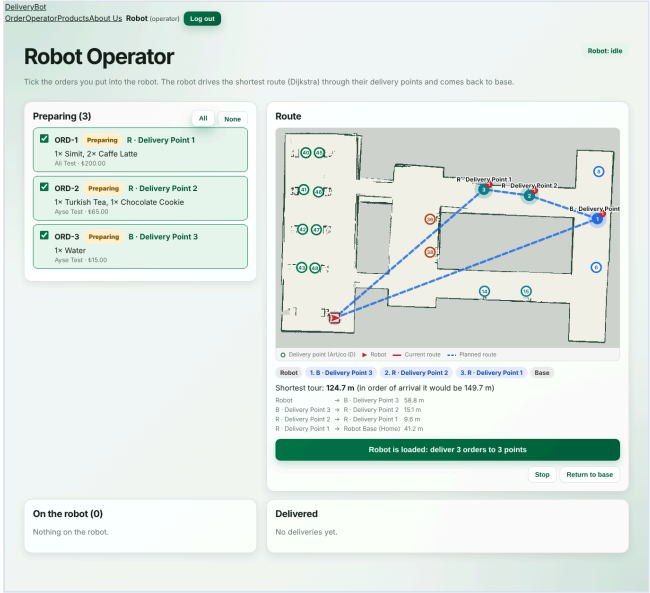


Figure 5. Web operator page: the same planner behind `/robot/plan` shows the shortest tour and its saving before the robot starts.

6. Results in the simulation

- campus_area offline test: stops right 1, left 4, right 3, left 1, end right 4 – optimal tour 173.8 m against 377.5 m in the given order; the robot drove 173 m and stopped 0.11–0.15 m from every pad.
- The navigator went around a box placed in the corridor (stop → obstacle added → replan).
- From the web app the operator's preview and the driven tour use the same code, so the distance shown before loading the robot is what the robot drives.

> **Summary.** Dijkstra on a clearance-weighted grid gives safe, centred paths and – run once per stop – all pairwise driving distances. Held-Karp turns those into the provably shortest delivery order in milliseconds for realistic loads (≤ 12 stops), saving about 40 % of the driving in the six-stop example.